

Bynry Backend Engineering Intern Case Study Submission

Submitted By: **Abhinav Shingote**
Role Applied: Backend Engineering Intern
Technology Chosen: Python Flask + SQLAlchemy

Part 1 - Code Review & Debugging

After reviewing the provided product creation API, I noticed that the code is syntactically correct but may not behave reliably in an actual B2B inventory management system. My approach was to compare the code flow with the business requirements given in the problem statement and identify practical situations where this endpoint can fail.

The first issue is that no validation is performed on the incoming request data. The code directly accesses fields like name, sku, price, and warehouse_id. If any of these fields are missing or null, the API may throw an internal server error instead of returning a meaningful response to the client.

The second issue is that SKU uniqueness is not checked before inserting the product. Since SKU is expected to be unique across the platform, allowing duplicate SKU values can create confusion in product identification, supplier mapping, and inventory reports.

Another major issue is that the product record is directly associated with a single warehouse using warehouse_id. However, the additional context clearly mentions that products can exist in multiple warehouses. This means the current structure does not fully support the intended business model.

I also observed that the database commit is being done twice. The product is committed first, and inventory is committed second. If the first commit succeeds but the second fails, then the system will save a product without any inventory record, which leads to incomplete data and inconsistent stock management.

The endpoint also does not contain any exception handling. In case of database failure or invalid data type, the API may crash without rolling back the transaction properly.

Additionally, negative values for price and initial quantity are not checked. In a practical inventory platform, negative stock or negative product price should not be accepted because it can produce invalid reports.

Based on these observations, my correction approach was to keep the API simple but safer. I added request validation, duplicate SKU checking, basic value validation, and combined the product and inventory save flow into one commit so that partial data insertion can be reduced.

This approach does not make the endpoint unnecessarily complex, but it makes it more stable and closer to expected production behaviour.

Relevant correction snippet:

```
if 'name' not in data or 'sku' not in data or 'price' not in data:  
    return {"error": "required fields missing"}, 400
```

```
existing_product = Product.query.filter_by(sku=data['sku']).first()  
if existing_product:  
    return {"error": "SKU already exists"}, 400
```

```
db.session.add(product)  
db.session.add(inventory)  
db.session.commit()
```

The full corrected implementation has been added in the submitted GitHub repository.

Part 2 - Database Design

For the database design section, I first converted the given business requirements into separate real-world backend entities. Since StockFlow is a B2B SaaS inventory platform, I assumed that data should be organised in such a way that one company can manage multiple warehouses, one product can be stored in multiple warehouses, and supplier relationships can also be tracked independently.

Based on this understanding, I designed the following main tables:

1. Company

Stores the business client using the StockFlow platform.

Important fields:

- company_id
- company_name

2. Warehouse

Each company can have multiple warehouses in different locations.

Important fields:

- warehouse_id
- company_id
- warehouse_name
- location

This creates a one-to-many relationship between company and warehouse.

3. Product

Stores the master product information.

Important fields:

- product_id
- name
- sku
- price

SKU is kept unique because duplicate SKU values can create product confusion.

4. Inventory

This table acts as a bridge between product and warehouse.

Important fields:

- inventory_id
- product_id
- warehouse_id
- quantity

- threshold
- avg_daily_sales

I used a separate inventory table because the same product can exist in multiple warehouses with different quantities. This table also stores low stock threshold and average sales information which will later help in low stock alert generation.

5. Supplier

Stores supplier details for each company.

Important fields:

- supplier_id
- company_id
- supplier_name
- contact_email

6. ProductSupplier

This table maps which supplier provides which product.

Important fields:

- id
- product_id
- supplier_id

This is useful because one supplier can provide multiple products.

7. InventoryTransaction

This table is used to track stock movement over time.

Important fields:

- transaction_id
- inventory_id
- change_amount
- transaction_type
- created_at

This helps in maintaining inventory history whenever stock is added, sold, or adjusted.

Overall, this schema keeps product master data separate from warehouse quantity data, which makes the design more scalable and easier to maintain.

TABLE 1 — PRODUCT TABLE

Column Name	Data Type	Description	Constraint
product_id	Integer	Unique product ID	Primary Key
name	String	Product name	Not Null
sku	String	Unique SKU code	Unique
price	Float	Product price	Not Null

TABLE 2 — WAREHOUSE TABLE

Column Name	Data Type	Description	Constraint
warehouse_id	Integer	Warehouse ID	Primary Key
company_id	Integer	Owner company	Foreign Key
name	String	Warehouse name	Not Null
location	String	Warehouse location	Optional

TABLE 3 — INVENTORY TABLE

Column Name	Data Type	Description	Constraint
inventory_id	Integer	Inventory record ID	Primary Key
product_id	Integer	Linked product	Foreign Key
warehouse_id	Integer	Linked warehouse	Foreign Key
quantity	Integer	Available stock	Default 0
threshold	Integer	Low stock limit	Default 10
avg_daily_sales	Integer	Avg daily usage	Default 1

TABLE 4 — SUPPLIER TABLE

Column Name	Data Type	Description	Constraint
supplier_id	Integer	Supplier ID	Primary Key
company_id	Integer	Associated company	Foreign Key
name	String	Supplier name	Not Null
contact_email	String	Supplier email	Optional

TABLE 5 — PRODUCT SUPPLIER TABLE

Column Name	Data Type	Description	Constraint
id	Integer	Mapping ID	Primary Key
product_id	Integer	Linked product	Foreign Key
supplier_id	Integer	Linked supplier	Foreign Key

TABLE 6 — INVENTORY TRANSACTION TABLE

Column Name	Data Type	Description	Constraint
transaction_id	Integer	Transaction ID	Primary Key
inventory_id	Integer	Inventory reference	Foreign Key
change_amount	Integer	Quantity change	Not Null
transaction_type	String	Add / Remove / Adjust	Not Null
created_at	DateTime	Timestamp	Not Null

Simplified Relationship View:

Company → Warehouse
Product → Inventory ← Warehouse
Product → ProductSupplier ← Supplier
Inventory → InventoryTransaction

Questions I Would Ask the Product Team:

Since the requirements were intentionally incomplete, I would clarify some points before final production development:

1. Can one product have multiple suppliers or only one primary supplier?
2. Should low stock threshold be fixed manually or auto-calculated by product type?
3. How should recent sales activity be measured — last 7 days, 15 days, or 30 days?
4. Are bundled products sold independently or only as a package?
5. Is inventory transaction tracking needed for audit purposes only or also for reporting dashboards?

These questions are important because backend database design depends heavily on such business rules.

Why I Chose This Design:

I kept the schema normalized but not overly complex. My main focus was to separate product details, warehouse storage details, supplier mapping, and inventory movement into independent tables. This reduces duplicate data and makes future API queries easier, especially for stock alerts, warehouse reports, and supplier reorder operations.

Part 3 - API Implementation

For the final section, I implemented a low stock alert endpoint for a company. The main purpose of this API is to scan all warehouse inventory records belonging to a company, compare current stock with predefined threshold values, and return only those products that need urgent replenishment.

The required endpoint is:

GET /api/companies/{company_id}/alerts/low-stock

To build this API, I followed a simple backend flow:

Step 1:

Fetch all inventory records by joining Inventory, Product, and Warehouse tables for the requested company.

Step 2:

For each inventory item, compare current quantity with its threshold value.

Step 3:

If current quantity is lower than threshold, mark that item as a low stock alert.

Step 4:

Fetch supplier details using ProductSupplier and Supplier mapping so that reorder information can be shown.

Step 5:

Calculate approximate days until stockout using a simple formula:

$$\text{days_until_stockout} = \text{current_stock} / \text{average_daily_sales}$$

This gives a rough estimate of how quickly the warehouse may run out of stock.

Finally, all matching low stock items are returned in JSON format with warehouse and supplier information.

I intentionally kept the API readable and modular so that business rules can be changed later if needed.

Core Implementation Snippet:

```
inventories = db.session.query(Inventory, Product, Warehouse)\
    .join(Product, Inventory.product_id == Product.id)\
    .join(Warehouse, Inventory.warehouse_id == Warehouse.id)\
    .filter(Warehouse.company_id == company_id)\
    .all()

for inventory, product, warehouse in inventories:
    if inventory.quantity < inventory.threshold:
        alerts.append({
            "product_id": product.id,
            "product_name": product.name,
            "warehouse_name": warehouse.name
        })
```

Edge Cases Considered:

While implementing this API, I considered a few practical edge cases:

1. If a company has no warehouses, API should return zero alerts.
2. If average daily sales is zero, stockout days calculation should not divide by zero.
3. If supplier mapping is not available, API should still return product alert with supplier as N/A.
4. If warehouse has stock exactly equal to threshold, it can be treated as safe or alert depending on future business preference.

These checks make the API more usable in real scenarios.

Reason for This Approach:

My focus in this implementation was not only returning low stock products, but also making the response useful for immediate business action. That is why warehouse name, supplier contact, and approximate stockout duration are included together in one response.

Sample Response Format:

```
{
  "alerts": [
    {
      "product_id": 101,
      "product_name": "Widget A",
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
    }
  ]
}
```

```
    "supplier": "ABC Supplier"  
  },  
  ],  
  "total_alerts": 1  
}
```

Final Assumptions and Closing Notes

Since the case study requirements were intentionally incomplete, I made a few practical assumptions during implementation:

1. Every inventory record has a manually stored low stock threshold.
2. Average daily sales value is available in the inventory table for rough stockout prediction.
3. One product has one primary supplier mapping for reorder simplicity.
4. Warehouse belongs to only one company.
5. The focus of this submission is backend logic demonstration, not full production deployment.

Throughout this case study, I tried to keep my solution simple, readable, and aligned with the given business requirements. My intention was to demonstrate practical backend thinking, identify where production issues can occur, and then build a maintainable approach rather than writing an unnecessarily complex solution.

GitHub Repository Link

<https://github.com/AbhinavShingote/bynry-backend-case-study-abhinav>